

生产企业原材料的订购与运输

基于词典序 MILP/LP 与独立验证

摘 要

针对板材生产企业未来 24 周的原材料保障、采购成本、运输损耗与产能估计问题，本文基于 402 家供应商和 8 家转运商的 240 周历史记录，构建可解释的供应商评价与词典序 MILP/LP 决策框架。预处理严格区分“未订购且零供货”和“已订购但零供货”，以常规期望供货上限描述历史供应能力，并仅用非零运输记录估计损耗率。

问题一由产品等价供货能力、按单供货概率、稳定性和兑现率形成业务型加权评分；小幅单项权重扰动下前 50 名重合率不低于 98%，但等权时 S140 从第 3 名降至第 53 名，说明排序依赖业务偏好。问题二先闭合最少供应商数的上下界：能力最强的前 25 家仅覆盖 28,044.41 m³/周，低于 28,200 m³/周需求，而 26 家正式模型可行；随后在全部 402 家候选中只锁定供应商基数值，以相对采购成本和运输损耗为后续词典序目标，求得单承运商方案，周成本为 20,463.57，周损耗为 102.94 m³。

问题三按“C 类最少、总原料最少、损耗最少”求解，A、B、C 类周供货分别为 8,627.38、6,567.37 和 2,896.16 m³，但出现 305 家正订购供应商和 48 个拆分供应商-周。问题四得到模型预测最大周产能 33,335.93 m³，较基准提高 18.21%。独立复算显示三问硬约束违约数均为 0，Excel 回读、公式重算和 14 项故障注入全部通过。

压力测试中，需求增加 10% 时已找到可行解但未证明成本最优；关键供应商能力下降 10% 和全期中断情景均得到全局最优可行解，供应商数分别为 26 和 27。上述结论限定在历史统计与平稳参数假设内，不构成未来供货或长期产能保证。

关键词：供应商评价；混合整数线性规划；词典序优化；库存平衡；运输分配；敏感性分析

目 录

一、问题重述	3
1.1 问题背景	3
1.2 需要解决的问题	3
二、问题分析	4
2.1 问题一分析	4
2.2 问题二分析	4
2.3 问题三分析	5
2.4 问题四分析	5
三、模型假设	5
四、符号说明	6
五、数据预处理与供应商特征	7
5.1 数据语义与审计	7
5.2 分布特征与建模影响	8
六、问题一：供应商重要性评价	8
6.1 指标定义	8
6.2 评价结果	9
七、问题二：最少供应商与订购运输模型	11
7.1 订购、供货与能力覆盖	11
7.2 运输与库存约束	12
7.3 三阶段词典序目标	12
八、问题二求解结果	13
8.1 正式方案与透明基线	13
8.2 转运瓶颈与库存轨迹	14
九、问题三：原料结构与损耗优化模型	15
9.1 三阶段词典序目标	15
9.2 连续运输与求解边界	15
十、问题三求解结果	15
10.1 原料结构变化	15
10.2 供应商分散与拆分运输	16
十一、问题四：模型预测最大周产能	16
11.1 目标函数与动态库存下界	16
十二、问题四求解结果	17
12.1 产能构成与独立复算	17
12.2 从现有库存到稳态的两周爬坡	17
十三、敏感性与压力测试	18
十四、独立复算与约束验证	19
14.1 验证设计	19
14.2 Python 与 MATLAB 交叉验证	19
十五、模型评价	19
15.1 优点	19
15.2 局限性	19
15.3 应用边界与改进方向	20
十六、结论	20
十七、参考文献	20
十八、附录	21

一、问题重述

1.1 问题背景

某板材生产企业以 A、B、C 三类植物纤维原料组织生产。企业当前每周基准产能为 28,200 m³。生产 1 m³ 产品分别需要 A、B、C 类原料 0.60、0.66、0.72 m³。三类原料可以替代生产，但其单位产品消耗量和采购价格不同：若 C 类单位采购价格取 1，则 B、A 类相对价格分别为 1.1 和 1.2。

供应商的实际供货量可能高于或低于订货量，企业会收购其实际供货。原料由 8 家转运商承担运输，各转运商每周运输能力均为 6,000 m³，运输过程中存在损耗。为维持连续生产，企业希望库存尽可能不少于 2 周生产需求；在运输组织上，通常希望同一供应商同一周的货物尽量由一家转运商承担。

附件 1 给出 402 家供应商连续 240 周的订货量与供货量，附件 2 给出 8 家转运商同期的运输损耗率。供应商中 A、B、C 类分别有 146、134、122 家。题目要求利用历史记录评价供应商、制定未来 24 周订购和转运方案，并估计技术改造后的最大产能。

1.2 需要解决的问题

1. 供应商重要性评价。量化供应商的供货规模、供货发生概率、稳定性和订单兑现能力，建立可解释的综合评价方法，从 402 家供应商中筛选 50 家重要供应商，并分析排序对权重的敏感性。
2. 最少供应商与经济运输。在满足每周 28,200 m³ 产品需求、两周安全库存、供应能力和转运能力等约束的前提下，确定最少供应商数量；在该数量下制定未来 24 周订购方案与低损耗转运方案。
3. 原料结构与损耗优化。在保障生产的基础上尽量多使用单位产品耗量较低的 A 类原料、少使用 C 类原料，同时减少原料采购运输总量和运输损耗，并披露连续运输模型可能产生的供应商分散与拆分运输问题。
4. 最大可持续周产能估计。在全部供应商历史能力估计、转运能力和损耗约束下，求技术改造后的模型预测最大周产能，并给出与该产能一致的未来 24 周订购和运输方案。

四问由“候选识别”逐步推进到“最小规模决策”“结构优化”和“产能边界估计”。供应商评价只提供候选信息，不能代替后续优化模型中的可行性、成本、损耗和库存约束。

二、问题分析

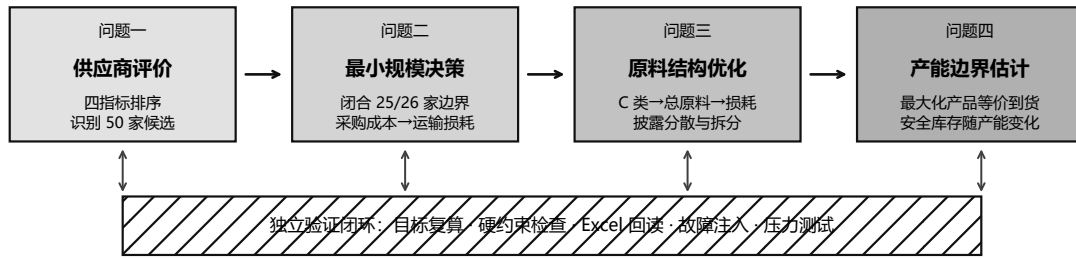


图 1 四问技术路线与验证闭环。供应商评价形成候选信息，随后依次完成最小规模、原料结构和产能边界优化。

2.1 问题一分析

供应商“重要性”不能等同于历史供货总量。若只按总量排序，长期被频繁下单的供应商会天然占优，也会把未订购周的零供货错误计为服务失败。因此，评价必须同时刻画规模、发生概率、波动和订单兑现四个维度，并明确零值的业务语义。

本文采用业务型线性加权评分，而不让样本离散度直接决定权重。能力反映生产保障上限，按单供货概率反映订单能否发生，稳定性刻画正供货状态下的波动，兑现率补充订单响应。评价权重具有业务主观性，排序只用于候选筛选，最终订购与运输决策仍由约束优化模型给出。等权条件下 S140 从第 3 名变为第 53 名，正说明“重要性”必须与权重语义一起解释。

数据处理中还必须区分两类零值：60,599 个“未订购且零供货”记录不应惩罚供应商；10,097 个“已订购但零供货”记录则是真实供货失败。若混合处理，两者会同时扭曲供货概率、正供货均值和兑现率。

2.2 问题二分析

问题二存在天然的优先级。企业首先需要回答“至少需要多少家供应商”，在最少数确定后才讨论经济订购，最后在采购成本不变的条件下降低运输损耗。若把三个目标直接加权相加，供应商数量、相对成本和损耗的量纲与数量级会干扰业务优先级，也不能保证先实现最少供应商。

因此采用词典序求解：先建立最小基数能力覆盖模型，得到供应商数量下界；再在正式运输、损耗和库存约束下证明该数量存在可行方案；随后只锁定最优供应商数量，依次最小化相对采购成本和运输损耗。能力最大的前 25 家在最有利损耗条件下仅提供 28,044.41 m³/周产品等价能力，仍低于需求；前 26 家的对应能力为 28,371.91 m³/周，且正式 MILP 可行，因而最少数量闭合为 26。

题目中的“尽量由一家转运商运输”在问题二中被落实为硬约束，使每个活跃供应商-周只对应一个承运商。该设置形成可直接执行的混合整数线性规划方案，但也增加二元变量规模。

2.3 问题分析

问题三的目标不同于问题二：采购价格不再是首要目标，题意强调多用 A、少用 C、减少原料总量并降低运输损耗。由于生产 1 m³ 产品所需 A 类原料最少，在 C 类用量固定后，最小化总原料量会自然偏向 A 类。因此将目标依次设为“C 类最少、总原料最少、运输损耗最少”，并用顺序锁定最优值的方式实现词典序优化。

若对全部可用供应商强制配置单一承运商，需要引入大量二元变量。既有硬单承运商路线在 80 秒内未返回可行解，其状态只能记为限时内状态未知，不能写成“不可行”。正式模型据此采用连续运输流，并将拆分次数与次要承运量作为软运营指标披露。

连续目标未设置供应商启用固定成本，因此出现 305 家正订购供应商并不违反已建约束，却会增加合同和管理复杂度。该结果必须作为模型边界保留，不能为了表面简洁而手工删除小额订购量；若要减少合作供应商，应另行加入启用变量和固定合作成本。

2.4 问题四分析

问题四不是把基准需求简单上调，而是在全部供应商能力、转运能力和损耗约束下，最大化每周损耗后可用于生产的产品等价到货量。A、B、C 三类原料先扣除运输损耗，再按单位产品耗量换算为统一产品等价量。

安全库存下界必须随产能同步变化。基准问题的两周安全库存为 56,400 m³ 产品等价量；当周产能变为决策变量后，初始库存与每周期末库存下界均应取该变量的两倍，不能继续固定为基准值。

最终 33,335.93 m³/周是历史能力估计与平稳损耗参数下的模型预测值，较 28,200 m³/周提高 18.21%。该数值用于刻画模型边界，不代表企业在未来波动、供应中断或合同变化下必然长期实现。

三、 模型假设

为使历史记录能够映射到未来 24 周决策，本文采用表 1 所列假设。假设只用于限定模型适用范围，不改变既有求解结果。

表 1 模型假设及其影响

编号	假设	说明与影响
A1	A、B、C 可替代生产	库存统一换算为可生产产品等价体积，原始订购与运输量仍使用原料单位。
A2	初始库存恰为两周需求	基准问题取 56,400 m³ 产品等价量；问题四随产能变量同步变化。
A3	未来供应能力采用历史期望上限	能力由非零供货均值与按单供货概率构成，是期望上限而非保证量。
A4	订货量按历史订单响应率反推	响应率截断到 1，避免依赖历史超额供货。
A5	损耗率采用非零历史记录均值	题面中损耗率 0 表示没有运输，不作为零损耗样本。
A6	不删除零值与极端值	官方数据无数值缺失和负值；IQR 异常只标记不剔除。
A7	原料量允许连续	问题二的承运商选择为二元；问题三、四采用连续运输流。
A8	采购成本为相对成本	C 类取 1.0，B、A 分别取 1.1、1.2，不能解释为人民币金额。
A9	未来参数平稳	供应能力、损耗率、价格和需求不随周次变化，单周最优结构可复制到 24 周。

单周—24 周口径：在需求、价格、供货能力和运输参数均保持平稳，且不存在周折扣、仓储上限和终端库存奖励时，各周优化结构相同。本文先求解代表性单周方案，再复制到未来 24 周，并依据库存平衡式逐周重建库存轨迹；正文报告成本和损耗均为周值，24 周总值为其 24 倍。供应商集合在 24 周内共享， 10^{-6} 是单周词典序目标锁定的数值容差。

这些假设中，A3 与 A9 对外推影响最大。历史常规期望供货上限不含未来概率区间，平稳参数也不描述季节性、趋势和突发中断。因此，后续的可行性与最优性结论均是“在给定参数和已建约束下”成立，而非对现实供应链的无条件保证。

数据审计显示，官方表格不存在数值缺失和负值；9,071 个正值原料记录超过 277 m^3 的 $Q_3 + 3IQR$ 阈值。由于大额供货可能正是识别核心供应商所需的信息，且无证据表明其为录入错误，故只标记而不删除或截尾。

四、符号说明

本文在原料体积、产品等价体积和相对成本之间保持严格区分。表 2 给出集合、参数和决策变量；下标 i 、 j 、 t 分别对应供应商、转运商和未来周次。

表 2 主要符号

符号	含义	单位
$i \in \mathcal{I}$	供应商索引， $ \mathcal{I} = 402$	-
$j \in \mathcal{J}$	转运商索引， $ \mathcal{J} = 8$	-
$t \in \mathcal{T}$	未来周次， $t = 1, \dots, 24$	周
$k(i)$	供应商 i 的原料类别	-
q_A, q_B, q_C	单位产品所需 A、B、C 类原料量	$\text{m}^3 \text{原料} / \text{m}^3 \text{产品}$
c_A, c_B, c_C	A、B、C 类原料相对采购单价	相对成本/ m^3
u_i	供应商 i 的常规期望供货上限	$\text{m}^3 / \text{周}$
p_i	供应商 i 在已下单周的非零供货概率	-
ρ_i	历史订单响应率，截断到 $[0, 1]$	-
ℓ_j	转运商 j 的预测损耗率	-

集合与参数均由题面或历史统计确定。 u_i 只表示规律下单条件下的历史期望能力，不应解释为未来每周保证供货量。

表 2（续） 主要符号

符号	含义	单位
U_j	转运商 j 的周运输能力，均为 6,000	$\text{m}^3/\text{周}$
z_i	是否启用供应商 i	0-1
o_{it}	第 t 周向供应商 i 的订购量	m^3
s_{it}	第 t 周供应商 i 的期望实际供货量	m^3
x_{ijt}	第 t 周供应商 i 经转运商 j 的发运量	m^3
y_{ijt}	问题二中是否由转运商 j 承运供应商 i 当周货物	0-1
r_{it}	损耗后接收的供应商 i 原料量	m^3
A_t	第 t 周损耗后到货的产品等价量	$\text{m}^3\text{产品}$
I_t	第 t 周末产品等价库存	$\text{m}^3\text{产品}$
D	周生产需求；问题四中为决策变量	$\text{m}^3\text{产品}/\text{周}$
$E(v, k)$	原料体积 v 按类别 k 换算的产品等价量	$\text{m}^3\text{产品}$

三类原料按单位产品消耗系数换算为产品等价量：

$$E(v, k) = \frac{v}{q_k}, \quad q_A = 0.60, \quad q_B = 0.66, \quad q_C = 0.72. \quad (1)$$

式 1 中， v 为原料体积， $k \in \{A, B, C\}$ 为原料类别， q_k 为生产 1 m^3 产品所需的该类原料体积， $E(v, k)$ 为可生产产品的等价体积。公式中的全部变量均已在表 2 定义。

五、数据预处理与供应商特征

5.1 数据语义与审计

附件 1 包含 402 家供应商连续 240 周的订货量与供货量，附件 2 包含 8 家转运商同期的运输损耗率。A、B、C 类供应商分别为 146、134、122 家。数据没有数值缺失和负值，预处理的关键不是删值，而是按题面区分不同零值的业务含义。

表 3 数据质量审计摘要

项目	订货量	供货量	损耗率	处理
数值缺失	0	0	0	无需填补
零值	60,599	10,097	479	保留并区分语义
负值	0	0	0	无异常
供货大于订货	-	16,703	-	题面允许，保留
正值 IQR 异常	9,071	合并统计	-	仅标记，不删除

共有 60,599 个“未订购且零供货”记录、10,097 个“已订购但零供货”记录和 25,784 个正供货记录。第一类记录不进入按单供货概率的失败计数，第二类记录则保留为真实履约失败。若不区分二者，会同时低估供应可靠性并高估正供货条件下的履约能力。

题面中损耗率为 0 表示当周末由该转运商承运，并非零损耗运输。因此，损耗率只在正记录上取均值。T1 至 T8 的预测损耗率依次为 1.90%、0.92%、0.19%、1.57%、2.89%、0.54%、2.08% 和 1.01%。正值原料量采用 $Q_3 + 3IQR$ 规则筛查，阈值为 277 m^3 ，共标记 9,071 个记录。大额供货可能恰是头部供应商能力的证据，故不截尾、不平滑；其未来可重复性留作模型边界。

5.2 分布特征与建模影响

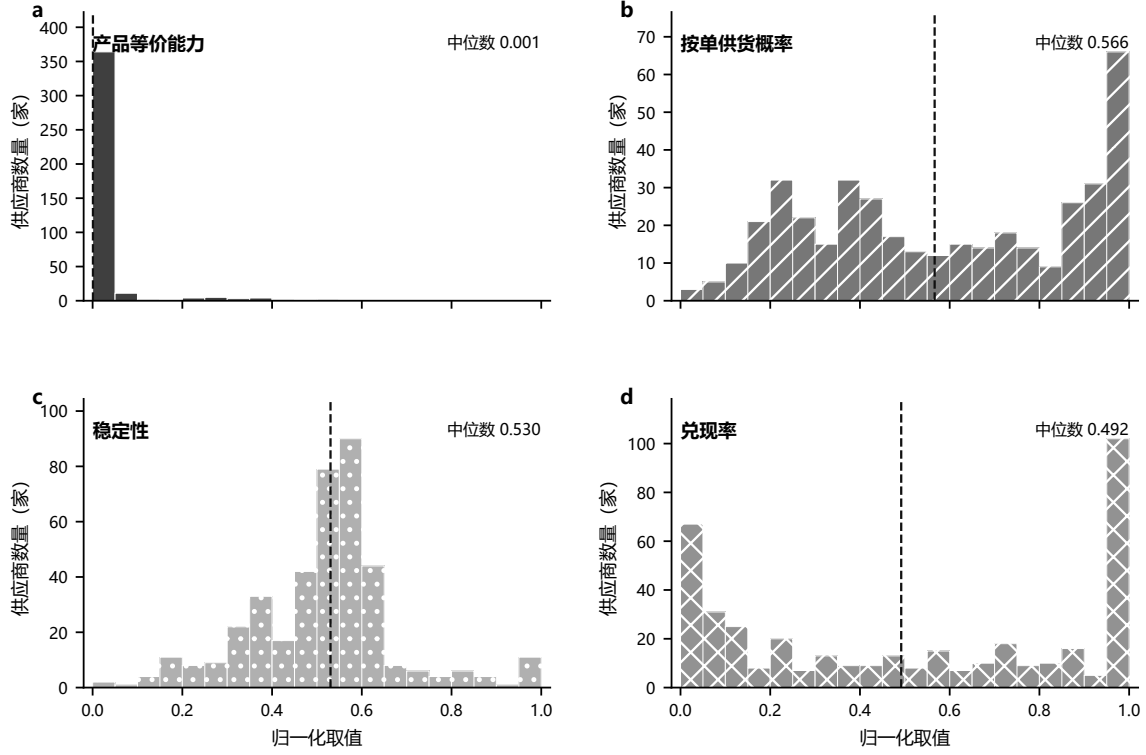


图 2 供应商四项评价指标分布。能力呈明显长尾，服务概率、稳定性和兑现率则更分散；虚线表示中位数。

图 2 显示，少数供应商贡献了主要产品等价能力，而规模较大的供应商并不必然具有同样高的按单供货概率、稳定性或兑现率。因此，问题一需要多指标评价；问题二至四仍须回到能力、运输和库存约束，而不能把综合排名直接当作可行方案。

数据处理保留全部官方记录。历史统计量用于条件期望估计，不被解释为未来保证值。

六、问题一：供应商重要性评价

6.1 指标定义

令历史订货量和供货量分别为 $O_{i\tau}$ 、 $S_{i\tau}$ ， $\tau = 1, \dots, 240$ ，已订购周集合为 $\Omega_i^O = \{\tau : O_{i\tau} > 0\}$ 。按单供货概率定义为

$$p_i = \frac{\sum_{\tau \in \Omega_i^O} 1(S_{i\tau} > 0)}{|\Omega_i^O|}. \quad (2)$$

式 2 只在实际下单周内计数。正供货均值和常规期望供货上限为

$$\bar{S}_i^+ = \frac{\sum_{\tau: S_{i\tau} > 0} S_{i\tau}}{\sum_{\tau} 1(S_{i\tau} > 0)}, \quad u_i = \bar{S}_i^+ p_i. \quad (3)$$

式 3 将发生正供货时的平均规模与按单供货概率结合。评价中的能力指标取产品等价能力 $C_i = \frac{u_i}{q_{k(i)}}$ 。稳定性使用正供货周变异系数的单调变换：

$$V_i = \frac{\sigma(S_{i\tau} \text{ mid } S_{i\tau} > 0)}{\bar{S}_i^+}, \quad G_i = \frac{1}{1 + V_i}. \quad (4)$$

兑现率定义为历史总供货量与总订货量之比，并截断到 1：

$$F_i = \min \left(\frac{\sum_{\tau} S_{i\tau}}{\sum_{\tau} O_{i\tau}}, 1 \right). \quad (5)$$

对 C_i, p_i, G_i, F_i 分别进行极差标准化:

$$\tilde{X}_i = \frac{X_i - \min_m X_m}{\max_m X_m - \min_m X_m}. \quad (6)$$

式 4、式 5 和 式 6 分别刻画供货波动、累计兑现程度及跨指标可比性。
最终综合重要性得分为

$$H_i = 0.50\tilde{C}_i + 0.25\tilde{p}_i + 0.15\tilde{G}_i + 0.10\tilde{F}_i. \quad (7)$$

式 7 以保障生产规模为首要业务偏好, 随后考虑订单能否发生、供货波动和累计兑现程度。权重不是统计意义上的客观真值, 因此排序必须与敏感性分析共同解释。多属性加权评分的思想可追溯至加性效用与多属性决策方法 [1-2]。

6.2 评价结果

表 4 重要性得分前 10 名供应商

排名	供应商	类型	综合得分
1	S229	A	0.900
2	S361	C	0.801
3	S140	B	0.706
4	S201	A	0.702
5	S275	A	0.673
6	S108	B	0.670
7	S329	A	0.669
8	S340	B	0.663
9	S282	A	0.652
10	S131	B	0.620

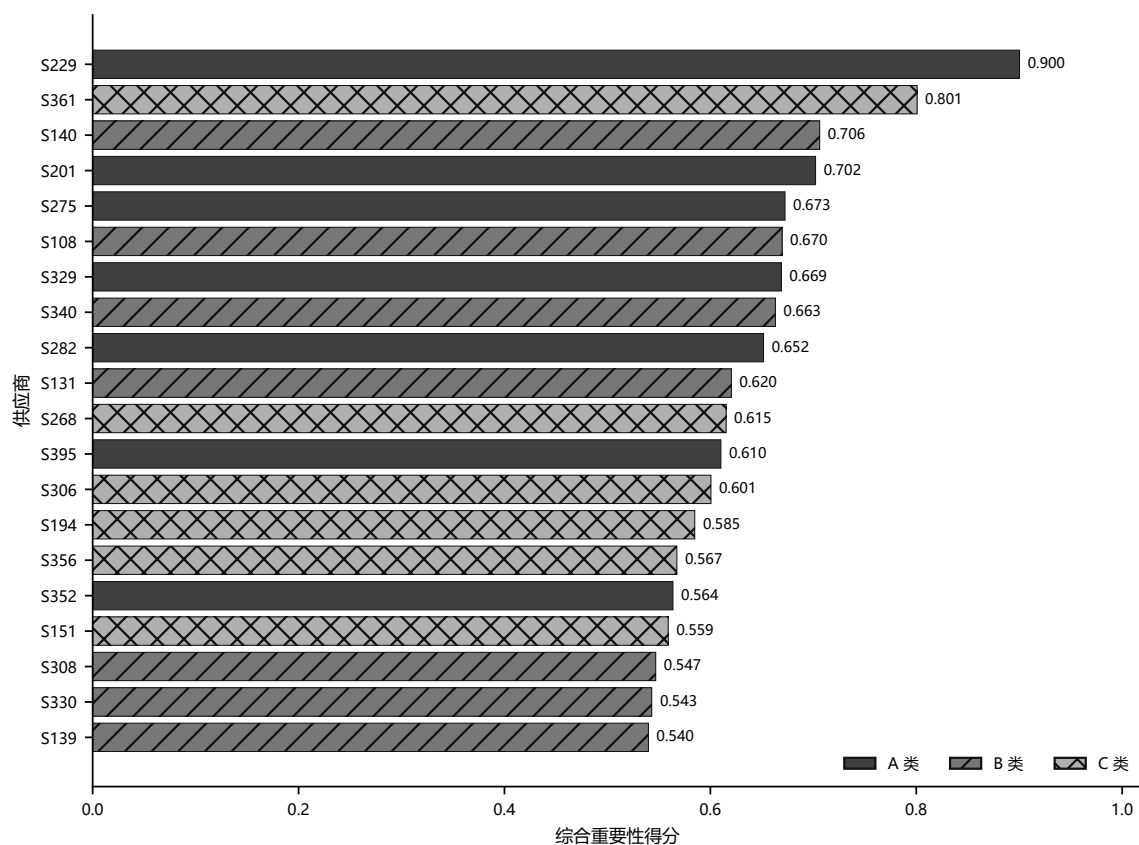


图 3 基准业务权重下综合得分前 20 名。灰度与填充纹理区分原料类别，排序表示给定权重下的候选优先级。

S229 和 S361 在各权重测试中保持前两名。S140 在基准权重下列第 3 名，但等权时降至第 53 名，表明其优势主要来自高权重的能力指标。

表 5 排名权重敏感性

权重方案	前 10 重合	前 50 重合率	Spearman 系数	S140 名次
当前权重	10/10	100%	1.000	3
单项相对扰动 $\pm 10\%$	至少 9/10	至少 98%	至少 1.000	3 或 4
等权	7/10	82%	0.976	53

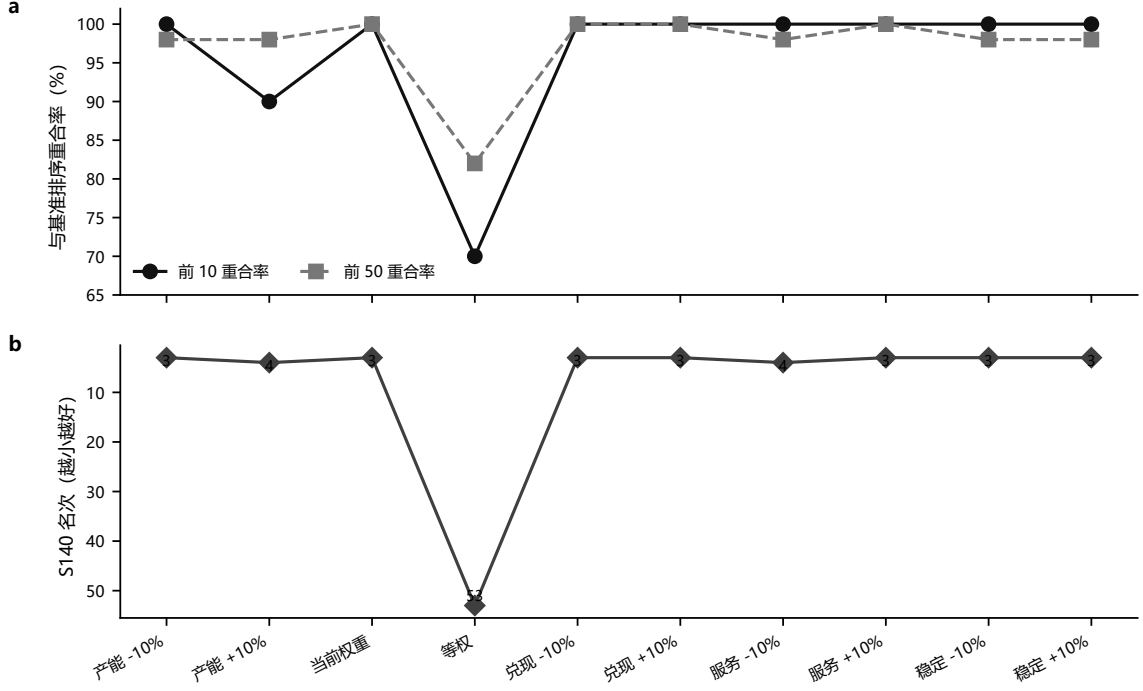


图 4 供应商排序的权重敏感性。小幅单项扰动下头部集合稳定，等权方案则暴露 S140 的名次依赖。

因此，前 50 名用于形成重点候选池，而非替代后续的可行性优化。完整前 50 名见结果附件 results/supplier_analysis.json，不与本文附录编号混用。

七、问题二：最少供应商与订购运输模型

问题二含供应商选择与单承运商二元变量，属于混合整数线性规划；问题三、四在连续流口径下属于线性规划 [3-4]。

7.1 订购、供货与能力覆盖

历史订单响应率截断到 $[0, 1]$ ，并用于由订购量反推期望实际供货量：

$$\rho_i = \min\left(\frac{\sum_{\tau} S_{i\tau}}{\sum_{\tau} O_{i\tau}}, 1\right), \quad s_{it} = \rho_i o_{it}. \quad (8)$$

式 8 中，采购成本按企业实际收购的 s_{it} 计量， o_{it} 只用于形成订购方案。以 z_i 表示是否选择供应商，并用最低预测损耗 $\ell_{\min} = \min_j \ell_j$ 构造必要能力覆盖模型：

$$\min \sum_{i \in \mathcal{I}} z_i. \quad (9)$$

$$\sum_{i \in \mathcal{I}} \frac{u_{i(1-\ell_{\min})}}{q_{k(i)}} z_i \geq D, \quad z_i \in \{0, 1\}. \quad (10)$$

式 9 最小化供应商基数，式 10 是正式模型的放松必要条件。将净产品能力记为 $b_i = \frac{u_{i(1-\ell_{\min})}}{q_{k(i)}}$ 并降序排列，则任意 25 家集合 S 满足 $\sum_{i \in S} b_i \leq \sum_{r=1}^{25} b_{(r)} = 28,044.41 < 28,200$ 。加入第 26 家后覆盖量为 28,371.91 $\text{m}^3/\text{周}$ ，且正式模型存在可行解。因此最少数量为 26，而不是一次启发式返回值。该数量只对应基准需求下的数学下界，不表示企业推荐维持的长期合作规模。

能力排序只用于构造下界证明和透明基线，不用于固定正式候选名单。正式求解保留全部 402 家正能力供应商；代表性单周模型包含 3,216 个运输流变量和 3,216 个二元指派变量，供应商组合与转运分配同时由 MILP 决定。

7.2 运输与库存约束

供应商期望供货全部进入运输流：

$$s_{it} = \sum_{j \in \mathcal{J}} x_{ijt}. \quad (11)$$

以单周是否启用的转运指派变量定义供应商启用变量：

$$z_i = \sum_{j \in \mathcal{J}} y_{ijt}, \quad z_i \in \{0, 1\}. \quad (12)$$

由于式 15 限制 $\sum_j y_{ijt} \leq 1$ ， z_i 与代码中供应商 i 的指派变量之和在数学上等价，无需额外引入 402 个求解变量。供应能力限制为

$$0 \leq s_{it} \leq u_i z_i. \quad (13)$$

每家转运商的周运输能力为

$$\sum_{i \in \mathcal{I}} x_{ijt} \leq U_j = 6000, \quad \forall j, t. \quad (14)$$

问题二把同一供应商同一周由一家转运商承运作为硬约束：

$$\sum_j y_{ijt} = z_i, \quad \sum_j y_{ijt} \leq 1, \quad 0 \leq x_{ijt} \leq u_i y_{ijt}, \quad y_{ijt} \in \{0, 1\}. \quad (15)$$

损耗后接收的原料量为

$$r_{it} = \sum_j x_{ijt} (1 - \ell_j). \quad (16)$$

当周产品等价到货量为

$$A_t = \sum_i \frac{r_{it}}{q_{k(i)}}. \quad (17)$$

库存按产品等价量统一记账，其守恒关系为

$$I_t = I_{t-1} + A_t - D. \quad (18)$$

基准问题的初始库存和安全库存下界为

$$I_0 = 2D, \quad I_t \geq 2D, \quad t = 1, \dots, 24. \quad (19)$$

式 11 至 式 19 分别约束供货守恒、历史能力、运输能力、单承运商、损耗后接收、产品换算和库存安全。库存平衡采用经典确定性库存结构 [6]，但能力与损耗参数来自本文的数据语义处理。

7.3 三阶段词典序目标

第一阶段在全部候选中求得最优供应商基数 $K^* = 26$ 。第二阶段只加入

$$\sum_i z_i = K^*, \quad (20)$$

并在所有 K^* 家组合中最小化相对采购成本：

$$Z_1 = \min \sum_{t,i} c_{k(i)} s_{it}. \quad (21)$$

第三阶段最小化运输损耗：

$$Z_2 = \min \sum_{t,i,j} \ell_j x_{ijt}. \quad (22)$$

式 21 和 式 22 分别对应第二、第三优先级。
设第二阶段最优值为 Z_1^* ，第三阶段加入

$$\sum_{t,i} c_{k(i)} s_{it} \leq Z_1^* + 10^{-6}. \quad (23)$$

式 20 只锁定供应商数量，不锁定具体名单；式 23 只锁定成本值，不锁定完整第二阶段解。 10^{-6} 是代表性单周成本目标的绝对容差，用于吸收浮点误差，同时保持采购成本优先于损耗。顺序求解符合词典序多目标优化的定义 [5]，也避免人为设置大权重造成尺度失真。

问题二采用完整候选空间混合整数线性规划。供应商数量、采购成本和运输损耗依次优化，后续阶段只锁定前序目标值。

八、问题二求解结果

8.1 正式方案与透明基线

未来参数按历史统计量设为平稳，模型没有周折扣、仓储上限或终端库存奖励，因此代表性单周最优方案可复制为 24 周方案，并按 式 18 逐周重建库存。正式模型在全部 402 家候选中先锁定最优基数 26，再优化成本与损耗。该全候选空间结果不是对能力排序候选的文字重述，而是在全部可行的 26 家供应商组合中证明了成本最优，并在成本最优解集中证明了运输损耗最优。为检验全局优化的实际增益，本文设置满足全部硬约束的成本优先单承运商贪心基线；该基线逐个供应商补足需求，但不保证跨供应商和转运商的全局最优。

表 6 透明基线 with 问题二正式模型对比

指标	透明基线	词典序 MILP	变化
能力覆盖数	26	26	0
列入供应商数	30	26	-4
正供货供应商数	29	26	-3
周期望供货量/m ³	18,467.98	18,378.94	-89.04
周相对采购成本	20,465.54	20,463.57	-0.01%
周运输损耗/m ³	103.80	102.94	-0.83%
产品等价到货/m ³	28,200.00	28,200.00	0

两种方案均使每周产品等价到货量恰为 28,200 m³，成本和损耗差异不是少供货造成。正式模型把正供货供应商从 29 家压缩为已证明最少的 26 家，并在全部 26 家组合中取得成本最优，再在成本最优集合中取得损耗最优；采购成本下降 0.01%，损耗下降 0.83%。

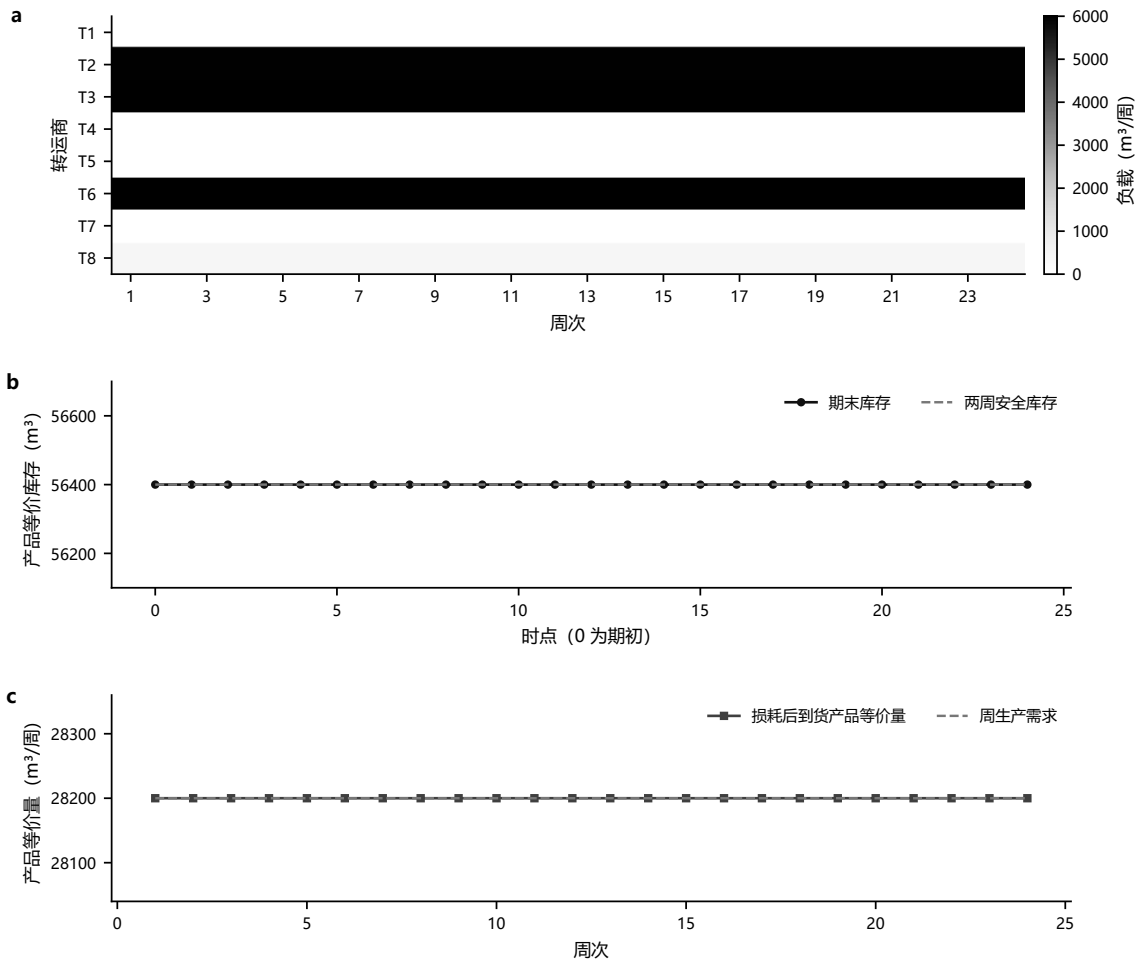


图 5 问题二的转运商负载、库存与到货量。T3、T6 达到能力上限，库存始终贴合两周安全下界。

8.2 转运瓶颈与库存轨迹

表 7 问题二各转运商损耗率与最大周负载

转运商	预测损耗率	最大周负载/m³	负载率
T1	1.90%	0.00	0.00%
T2	0.92%	5,956.59	99.28%
T3	0.19%	6,000.00	100.00%
T4	1.57%	0.00	0.00%
T5	2.89%	0.00	0.00%
T6	0.54%	6,000.00	100.00%
T7	2.08%	0.00	0.00%
T8	1.01%	422.35	7.04%

T3、T6 达到 6,000 m³/周上限，T2 负载率为 99.28%，说明低损耗运输资源是主要瓶颈。624 个活跃供应商-周均满足式 15，拆分数为 0。每周到货量恰好补足生产消耗，故期末库存维持在 56,400 m³安全下界。平稳轨迹是参数假设的结果，不意味着未来实际供货没有周际波动。

九、 问题三：原料结构与损耗优化模型

9.1 三阶段词典序目标

问题三保留式 8、式 11 至式 19 的供货、运输和库存关系，不再限制最少供应商数，也不把采购价格作为第一目标。第一阶段最小化 C 类原料期望供货量：

$$Z_3 = \min \sum_t \sum_{i:k(i)=C} s_{it}. \quad (24)$$

在锁定 Z_3^* 后，第二阶段最小化总原料量：

$$Z_4 = \min \sum_{t,i} s_{it}. \quad (25)$$

由于 $q_A < q_B < q_C$ ，在 C 类用量已最小的条件下，式 25 会优先使用单位产品耗料更少的 A 类。第三阶段在 $Z_3 \leq Z_3^* + 10^{-6}$ 、 $Z_4 \leq Z_4^* + 10^{-6}$ 下最小化运输损耗：

$$Z_5 = \min \sum_{t,i,j} \ell_j x_{ijt}. \quad (26)$$

三层目标按式 24 至式 26 顺序求解，分别对应“尽量少 C”“尽量少总原料”和“尽量少损耗”。每一层只在前序最优集内继续优化，因此不需要人为折中系数。

9.2 连续运输与求解边界

问题三采用连续运输流，不设置式 15 的二元指派。题面表述为“尽量由一家转运商承担”，连续 LP 能给出明确最优性状态；既有全网硬单承运商模型在 80 秒内没有获得可行解，只能记录为限时内状态未知，不能写成数学不可行，也没有可与当前 LP 比较的损耗最优值。

连续流允许少量拆分，并可能产生许多小额订单。本文不在求解后手工合并或删除，因为这会改变式 24 至式 26 的目标值，或使主承运商超过式 14 的容量上限。运营集中度因此作为结果边界单独披露。

十、 问题三求解结果

10.1 原料结构变化

表 8 问题三周均原料使用结构

原料	期望供货量/m ³	损耗后到货量/m ³	产品等价量/m ³
A	8,627.38	8,601.93	14,336.55
B	6,567.37	6,519.60	9,878.18
C	2,896.16	2,869.39	3,985.27
合计	18,090.91	17,990.92	28,200.00

与问题二相比，A 类周供货由 7,702.69 m³ 增至 8,627.38 m³，C 类由 5,235.34 m³ 降至 2,896.16 m³，总原料量由 18,378.94 m³ 降至 18,090.91 m³。三类损耗后产品等价量之和仍为 28,200 m³/周，因此结构变化没有牺牲生产需求。

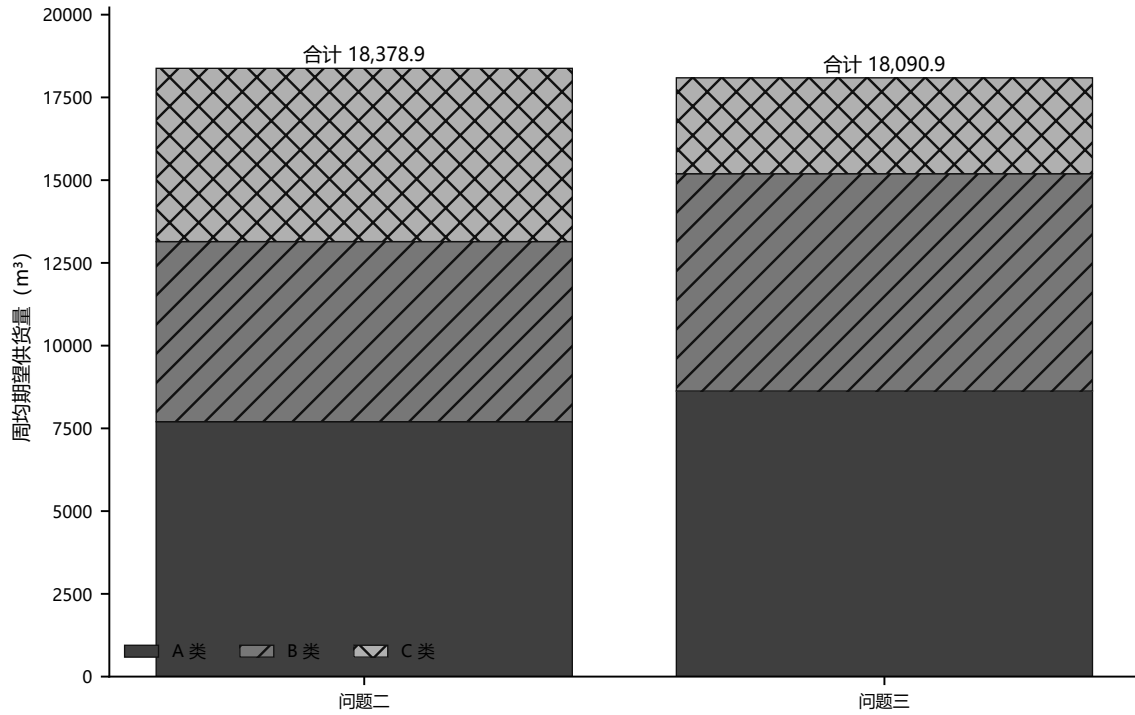


图 6 问题二与问题三的周均原料结构。问题三提高 A 类用量、压低 C 类用量，并减少总原料体积。

10.2 供应商分散与拆分运输

表 9 问题三供应商分散性与拆分运输

指标	数值	解释
正订购供应商数	305 家	连续 LP 未惩罚启用数量
最小单供应商周订购量	1.02 m³	不是容差附近伪非零
$P_{10}/\text{中位数}/P_{90}$	1.69/5.43/134.49	小额合作方较多
小于 10 m³ 的供应商	188 家	占正订购供应商 61.64%
小于 100 m³ 的供应商	268 家	占正订购供应商 87.87%
拆分供应商-周	48 / 7,320	发生率 0.66%
次要承运商运输量占比	5.06%	体积影响不能忽略
单周最多承运商数	2 家	拆分有限但真实存在

305 家供应商不是推荐合作规模，而是目标中没有固定启用成本的结果。拆分发生次数仅占 0.66%，但拆分周涉及 16.84% 的总运输量，次要承运商承担 5.06%；因此只报告次数会低估调度影响。若企业要求更集中、严格单承运商的方案，应在新模型中加入启用成本、最小订单量或拆分惩罚，并重新求解，而不能事后删单。

十一、 问题四：模型预测最大周产能

11.1 目标函数与动态库存下界

问题四使用所有具有常规期望供货上限的供应商，保留式 13、式 14、式 16 和式 17 的能力、运输、损耗与换算关系，并将周产能 D 作为决策变量：

$$\max_{x,D} D, \quad D = \sum_{i,j} \frac{x_{ij}(1 - \ell_j)}{q_{k(i)}}. \quad (27)$$

24 周库存下界随产能同步变化：

$$I_0 = 2D, \quad I_t = I_{t-1} + A_t - D, \quad I_t \geq 2D. \quad (28)$$

式 27 最大化稳定周流量，式 28 则避免把一次性初始库存释放误认为可持续产能。平稳方案满足 $A_t = D$ ，库存维持在 $2D$ ；若仍固定使用基准安全库存 $56,400 \text{ m}^3$ ，就会低估产能提高后的缓冲需求。因此，该目标是企业已提前建立 $2D$ 安全库存后的确定性稳态上界；从现有库存立即实施时还需单独安排过渡期。

该模型是连续 LP。其上界受供应商历史期望能力、各转运商 $6,000 \text{ m}^3/\text{周}$ 容量、预测损耗率和三类原料换算系数共同决定，不是简单汇总全部历史供货峰值。

十二、 问题四求解结果

12.1 产能构成与独立复算

表 10 问题四模型预测最大周产能复算

原料	损耗后到货量/ m^3	换算系数	产品等价量/ m^3
A	8,601.93	$\div 0.60$	14,336.55
B	6,519.60	$\div 0.66$	9,878.18
C	6,567.26	$\div 0.72$	9,121.20
合计	21,688.79	-	33,335.93

三类产品等价量相加得到 $33,335.93 \text{ m}^3/\text{周}$ ，比基准 $28,200 \text{ m}^3/\text{周}$ 增加 $5,135.93 \text{ m}^3/\text{周}$ ，增幅为 18.21%。相应两周安全库存为 $66,671.85 \text{ m}^3$ 产品等价量。

12.2 从现有库存到稳态的两周爬坡

企业当前库存为 $56,400 \text{ m}^3$ ，低于稳态安全库存 $66,671.85 \text{ m}^3$ ，尚需补足 $10,271.85 \text{ m}^3$ 。维持基准生产时，每周按最大到货方案采购可净积累 $5,135.93 \text{ m}^3$ ，故恰需 2 周完成补库。一个保守的 24 周实施方案为：第 1–2 周生产 $28,200 \text{ m}^3$ 、按 $33,335.93 \text{ m}^3$ 到货并逐周积累库存；第 3–24 周生产与到货均为 $33,335.93 \text{ m}^3$ ，进入稳态。该过渡方案的 24 周平均产量为 $32,907.93 \text{ m}^3/\text{周}$ ，较基准提高 16.69%。

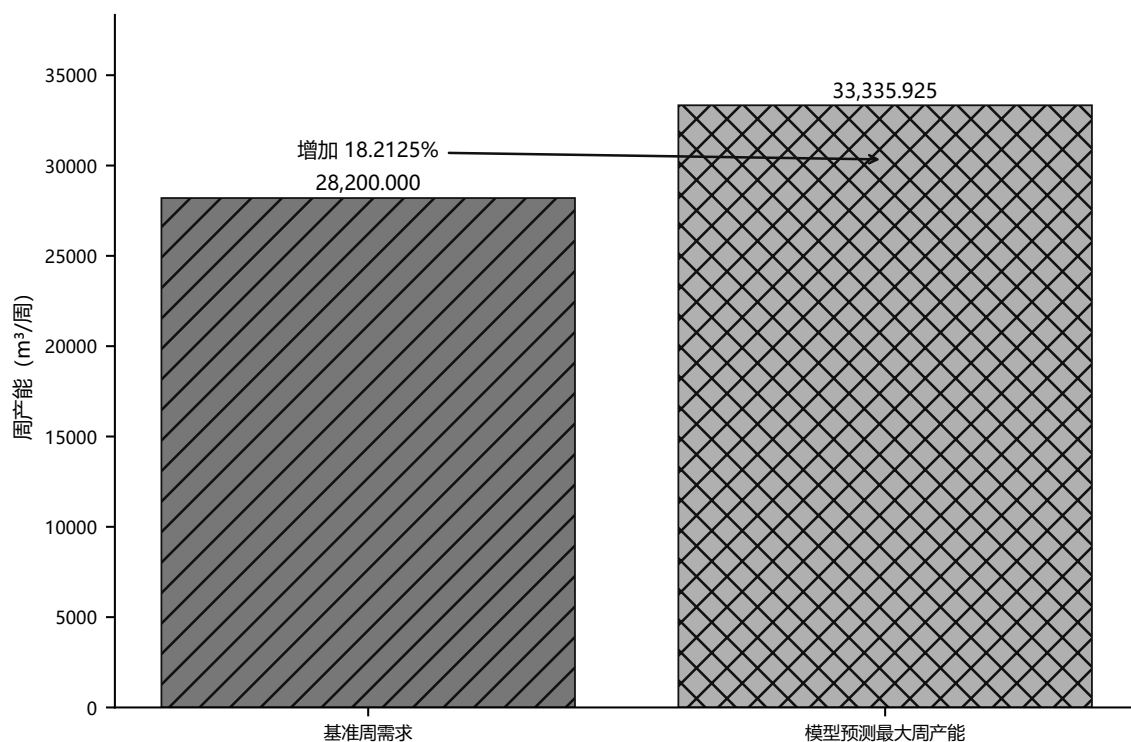


图 7 基准周需求与模型预测最大周产能。增幅为 18.21%，阴影范围用于强调两者差额。

低损耗转运能力仍是关键限制。T3、T6 达到或接近上限，T8 的负载增至 3,826.52 m³/周，说明扩大原料供给后还需启用次优运输通道。继续提高产能不能只增加供应商能力，还需同步改善低损耗转运资源。
 33, 335.93 m³/周是历史统计与平稳参数条件下、已提前建立相应安全库存后的确定性稳态上界，不是企业从当前库存即可实现或未来可长期保证的产能。

十三、 敏感性与压力测试

权重敏感性已在表 5 和图 4 中说明。对问题二，需求、损耗率和关键供应商能力情景均重新求解，而不是把原方案机械代入新参数。敏感性分析用于识别结论依赖的参数与失效方向 [7]，不等于证明所有扰动下均稳健。
 表 11 问题二压力测试

情景	状态	供应商数	周相对成本	解释
需求 -10%	可行	21	18,412.65	所需供应商明显减少
基准需求	可行	26	20,463.57	正式方案
需求 +10%	可行未证最优	41	22,521.70	基数已证；成本阶段限时
损耗率 × 0.8	可行	26	20,441.16	成本略降
损耗率 × 1.2	可行	26	20,486.31	仍可行但代价上升
关键能力 -10%	最优可行	26	20,463.98	替代组合吸收能力下降
关键供应商全期中断	最优可行	27	20,462.81	增加 1 家后恢复可行

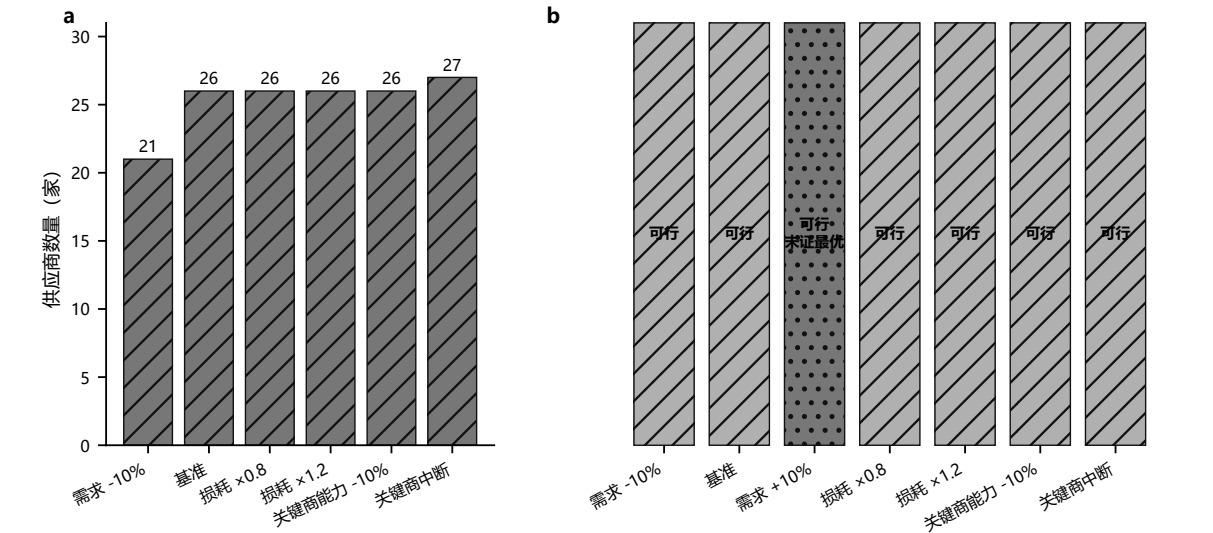


图 8 问题二压力测试。完整最优情景显示供应商数；需求增加 10% 单独标记为已找到可行解但未证明成本最优。

需求增加 10% 时，供应商基数阶段已证明最少需 41 家；采购成本阶段在 60 秒内获得周相对成本 22,521.70 的可行 incumbent，当前下界为 22,521.70，相对 MIP 间隙为 7.42e-9，但尚未完成最优性证明。因此不能写成全局最优或不可行。关键供应商能力下降 10% 和全期中断在完整模型中均得到全局最优可行解，分别需要 26 和 27 家供应商，说明开放全部候选后可通过替代组合恢复可行。实际部署仍应保留备用供应商，并在需求增长前扩充低损耗运输能力。

十四、独立复算与约束验证

14.1 验证设计

Python 正式求解采用 SciPy/HiGHS 的 MILP/LP 接口 [8-11]。独立检查器不导入求解器模块，只读取官方材料、锁定参数、原始决策变量和输出表，重新计算供应能力、订单响应、运输损耗、到货换算、库存与目标值。这样降低了“求解器与验证器共享同一错误公式”的风险。

表 12 正式结果验证摘要

验证项目	问题二	问题三	问题四	结论
独立约束检查次数	135,961	136,240	136,337	全部通过
硬约束违约数	0	0	0	通过
目标复算绝对误差	0	0	0	通过
Excel 决策单元回读	86,832	86,832	86,832	最大误差 0
故障注入	合并 14 项	合并 14 项	合并 14 项	14/14 检出

问题二相对采购成本复算为 491,125.620530，问题三 24 周 C 类原料复算为 69,507.840621 m³，问题四 24 周产品等价到货复算为 800,062.211085 m³，均与报告值零误差。结果工作簿 A、B 回读逐格比较正式决策与写入值；LibreOffice 重算后无公式错误或空缓存。14 项故障覆盖负订货、能力超限、库存守恒篡改、转运超载、多承运商、损耗和目标篡改等，全部由预期检查器拒绝。

14.2 Python 与 MATLAB 交叉验证

本文以 Python/HiGHS 作为正式优化实现，并使用 MATLAB 对供应商评价、词典序规划、目标值和硬约束进行独立复现。两种实现的主要目标值和聚合决策在预设容差内一致。MATLAB 直接读取官方 Excel，采用 intlinprog 求解问题二、linprog 求解问题三和问题四；它不调用 Python，也不把 Python 中间结果作为优化输入。前 10、前 50 排名完全一致，评分最大误差为 1.11×10^{-16} ，全部连续量的最大跨语言误差为 3.21×10^{-7} ，低于 10^{-6} 验收阈值；两端硬约束违约均为 0。具体版本、命令和退出状态见附录 E、F。验证回答的是“决策是否满足本文定义的模型”，不证明历史期望能力会在未来逐周实现，也不替代对价格、仓容和供应中断概率的现场核验。

十五、模型评价

15.1 优点

第一，模型严格区分订购量、期望供货量、发运量、损耗后接收量和产品等价库存，避免量纲混用。第二，四项供应商指标显式区分未订购零供货与已订购零供货，且通过权重敏感性限定排名解释。第三，问题二以 25 家能力不足和 26 家正式模型可行闭合最小基数证明。第四，三问均采用顺序锁定的词典序目标，业务优先级可解释。第五，Python 独立检查、MATLAB 跨语言复现、Excel 回读和故障注入从不同路径核查同一组目标与约束。

15.2 局限性

供应商权重包含业务判断，等权时 S140 的名次变化说明排序不是绝对客观。常规期望供货上限只描述历史统计，不能代表未来保证能力；损耗率采用正记录条件均值，未建立时间序列或区间不确定模型。题面没有仓储上限、绝对采购价、运输费用和供应中断概率，因此模型不能评价现金流、仓容与概率服务水平。

问题三连续 LP 没有供应商启用成本, 导致 305 家供应商和大量小额订单; 问题三、四允许少量拆分运输。若企业坚持硬单承运商, 应建立更大 MILP 并重新求解。平稳参数还使 24 周方案在时间维度重复, 未识别季节性、提前期和滚动预测误差。安全库存设为确定性的两周需求下界, 也没有随需求方差和服务水平变化。

15.3 应用边界与改进方向

该框架可扩展到多品类替代原料、运输容量分配和安全库存联动问题。若新增未来价格、仓容、固定管理成本、最小订单量、供应中断概率或需求分布, 可进一步构建时变、稳健或情景优化模型。但这些扩展会改变目标体系和结果, 不能被当作对当前锁定方案的事后修补。

十六、 结论

本文在统一的数据语义下完成供应商评价、最少供应商选择、订购运输优化、原料结构优化和最大产能估计。问题一的基准权重下, S229、S361 和 S140 位列前三; 小幅权重扰动保持头部集合稳定, 但等权使 S140 从第 3 名降至第 53 名, 说明排序服务于给定业务偏好。

问题二证明基准生产至少需要 26 家供应商, 并在全部 402 家候选中只锁定该基数后继续优化。正式单承运商词典序 MILP 的周相对采购成本为 20,463.57, 周损耗为 102.94 m³; 相对透明基线, 成本下降 0.01%, 损耗下降 0.83%。问题三把 C 类周供货压缩到 2,896.16 m³, 总原料量降至 18,090.91 m³/周, 但产生 305 家正订购供应商和少量拆分运输。问题四得到 33,335.93 m³/周的确定性稳态上界, 较基准提高 18.21%; 从现有库存保守实施时先用 2 周补库, 24 周平均产量为 32,907.93 m³/周。

Python 独立复算、MATLAB 交叉复现、硬约束检查、Excel 回读、公式重算和 14 项故障注入均通过; Python-MATLAB 最大数值误差为 3.21×10^{-7} 。压力测试显示: 需求增加 10% 时已找到可行解但未证明成本最优; 关键供应商能力下降 10% 和全期中断均可通过完整候选空间中的替代组合恢复可行。因而本文结论只在历史统计、平稳参数和已建模约束下成立, 不构成未来供货、真实最优合作规模或长期保证产能的承诺。

十七、 参考文献

- [1] Fishburn P C. Letter to the editor: Additive utilities with incomplete product sets: application to priorities and assignments. *Operations Research*, 1967, 15(3): 537-542. DOI: 10.1287/opre.15.3.537.
- [2] Hwang C L, Yoon K. *Multiple Attribute Decision Making: Methods and Applications*. Berlin: Springer, 1981. DOI: 10.1007/978-3-642-48318-9.
- [3] Bertsimas D, Tsitsiklis J N. *Introduction to Linear Optimization*. Belmont, MA: Athena Scientific, 1997.
- [4] Nemhauser G L, Wolsey L A. *Integer and Combinatorial Optimization*. New York: Wiley, 1988. DOI: 10.1002/9781118627372.
- [5] Miettinen K. *Nonlinear Multiobjective Optimization*. Boston: Springer, 1998. DOI: 10.1007/978-1-4615-5563-6.
- [6] Porteus E L. *Foundations of Stochastic Inventory Theory*. Stanford: Stanford University Press, 2002. DOI: 10.1515/9781503619883.
- [7] Saltelli A, Ratto M, Andres T, et al. *Global Sensitivity Analysis: The Primer*. Chichester: Wiley, 2007. DOI: 10.1002/9780470725184.

[8] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 2018, 10(1): 119-142. DOI: 10.1007/s12532-017-0130-5.

[9] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 2020, 17(3): 261-272. DOI: 10.1038/s41592-019-0686-2.

[10] SciPy Developers. scipy.optimize.milp official documentation. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.milp.html>, accessed 2026-07-14.

[11] HiGHS Development Team. HiGHS: high-performance software for linear optimization. <https://highs.dev/>, accessed 2026-07-14.

十八、 附录

18.1 附录 A: Python 文件清单

正式 Python 工件共 14 个源文件、4,151 行。表 13 按职责列出文件；论文只嵌入核心求解文件，完整程序随可复现材料提供。

表 13 Python 源文件与职责

文件	职责
code/common/common.py	官方数据读取、锁定参数与序列化
code/solver/optimization.py	问题二 MILP；问题三、四 LP 正式求解
code/validator/independent_validator.py	目标与硬约束独立复算
code/analysis/artifacts.py	数据审计、评分和附件回填
code/analysis/generate_paper_assets.py	论文源数据与旧图工件
code/analysis/generate_typst_paper_figures.py	8 幅黑白 Typst 论文图
code/analysis/build_paper_docx.py	旧 DOCX 生成器，仅作历史工件
code/final_human_audit.py	权重、语义和人工审计证据
code/run_training.py	正式训练流程入口
code/tests/test_validator.py	验证器单元测试
code/common/__init__.py	公共包入口
code/solver/__init__.py	求解器包入口
code/validator/__init__.py	验证器包入口
code/analysis/__init__.py	分析包入口

18.2 附录 B: Python 核心源码

18.2.1 B.1 数据读取与预处理

文件: code/common/common.py、code/analysis/artifacts.py；作用: 读取官方 Excel、统一量纲并生成数据审计；输入: 附件 1、附件 2 与空白结果模板；输出: 标准化数组、材料清单和数据质量工件。

18.2.2 B.2 供应商评价与流程入口

文件: code/run_training.py；作用: 组织供应商评价、问题二至四求解、敏感性分析和结果导出；输入: 标准化官方数据与锁定假设；输出: 正式结果、验证工件、结果工作簿 A/B 和派生图。

18.2.3 B.3 问题二至四优化模块

文件: code/solver/optimization.py；作用: 使用 SciPy milp/HiGHS 构造供应能力、转运能力、单承运商、需求和词典序锁定约束；问题二为 MILP，问题三、四为连续 LP。输入: 供应商能力、损耗率、需求与模型参数；输出: 问题二至四决策变量、目标值和求解状态。以下保留该核心模块完整源码。

```

"""本轮问题二 MILP 与问题三、四 LP 的可审计求解器。

每个模型只使用本轮 ``common`` 中由官方材料计算出的参数；转运分配与订货量导出
均保留到完整精度，最终结果再交给独立检查器复算。
"""

from __future__ import annotations

```

```

import time
from dataclasses import dataclass
from typing import Any, Callable

import numpy as np
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import coo_matrix

from common.common import DEMAND_BASE, HORIZON, TOLERANCE, TRANSPORTER_CAPACITY, now_iso

@dataclass
class MilpRun:
    """保存一个 HiGHS 调用的解与可审计状态。"""

    x: np.ndarray
    status: dict[str, Any]

class OptimizationStatusError(RuntimeError):
    """携带结构化求解状态，避免压力测试把限时可行解误报为不可行。"""

    def __init__(self, message: str, status: dict[str, Any]) -> None:
        super().__init__(message)
        self.status = status

class ConstraintBuilder:
    """以稀疏三元组方式搭建线性约束，避免构造稠密大矩阵。"""

    def __init__(self, variable_count: int) -> None:
        self.variable_count = variable_count
        self.rows: list[int] = []
        self.columns: list[int] = []
        self.values: list[float] = []
        self.lower: list[float] = []
        self.upper: list[float] = []
        self._row = 0

    def add(self, indices: np.ndarray, values: np.ndarray, lower: float, upper: float) -> None:
        self.rows.extend([self._row] * len(indices))
        self.columns.extend(indices.astype(int).tolist())
        self.values.extend(values.astype(float).tolist())
        self.lower.append(float(lower))
        self.upper.append(float(upper))
        self._row += 1

    def linear_constraint(self) -> LinearConstraint:
        matrix = coo_matrix(
            (self.values, (self.rows, self.columns)),
            shape=(self._row, self.variable_count),
        ).tocsr()
        return LinearConstraint(matrix, np.array(self.lower), np.array(self.upper))

def _status(result: Any, start: float, objective_name: str, time_limit_seconds: float) -> dict[str, Any]:
    """抽取 SciPy/HiGHS 状态，避免把求解器内部对象写入结果。"""
    return {
        "solver": "SciPy HiGHS MILP",
        "solver_version": "SciPy 1.17.0 bundled runtime",
        "model_type": "mixed_integer_linear_program",
        "objective_name": objective_name,
        "start_time": now_iso(),
        "end_time": now_iso(),
        "runtime_seconds": time.perf_counter() - start,
        "time_limit_seconds": time_limit_seconds,
    }

```

```

        "status_code": int(result.status),
        "status_text": str(result.message),
        "feasible": bool(result.x is not None),
        "optimality_proven": bool(result.success),
        "incumbent": float(result.fun) if result.fun is not None else None,
        "best_bound": (
            float(result.mip_dual_bound)
            if getattr(result, "mip_dual_bound", None) is not None
            else None
        ),
        "mip_gap": float(getattr(result, "mip_gap", np.nan)) if getattr(result, "mip_gap", None) is not
None else None,
        "node_count": int(getattr(result, "mip_node_count", 0) or 0),
        "solver_objective": float(result.fun) if result.fun is not None else None,
    }

def _solve(
    objective: np.ndarray,
    builder: ConstraintBuilder,
    lower_bounds: np.ndarray,
    upper_bounds: np.ndarray,
    integrality: np.ndarray,
    objective_name: str,
    time_limit_seconds: float = 300.0,
) -> MilpRun:
    """运行一次有300秒上限的 HiGHS MILP, 并在无可行解时显式报错。"""
    start = time.perf_counter()
    result = milp(
        c=objective,
        integrality=integrality,
        bounds=Bounds(lower_bounds, upper_bounds),
        constraints=builder.linear_constraint(),
        options={"time_limit": time_limit_seconds, "mip_rel_gap": 0.0},
    )
    status = _status(result, start, objective_name, time_limit_seconds)
    if result.x is None:
        raise OptimizationStatusError(
            f"{objective_name} 未得到可行解: {status['status_text']}", status
        )
    if not result.success:
        raise OptimizationStatusError(
            f"{objective_name} 未证明最优: {status['status_text']}", status
        )
    return MilpRun(np.maximum(result.x, 0.0), status)

def _indices(supplier_count: int, transporter_count: int) -> tuple[np.ndarray, np.ndarray, int]:
    flow = np.arange(supplier_count * transporter_count, dtype=int).reshape(supplier_count,
transporter_count)
    assign = flow + supplier_count * transporter_count
    return flow, assign, supplier_count * transporter_count * 2

def _base_builder(
    data: dict[str, Any], candidate_indices: np.ndarray, loss_multiplier: float = 1.0,
    enforce_single_carrier: bool = True,
) -> tuple[ConstraintBuilder, np.ndarray, np.ndarray, dict[str, np.ndarray]]:
    """建立供应能力、转运能力与单一转运商的共同硬约束。"""
    capacities = data["regular_order_capacity"][candidate_indices].astype(float)
    raw_per_product = data["raw_per_product"][candidate_indices].astype(float)
    unit_cost = data["unit_cost"][candidate_indices].astype(float)
    losses = np.clip(data["loss_mean"].astype(float) * loss_multiplier, 0.0, 0.99)
    supplier_count = len(candidate_indices)
    transporter_count = len(losses)
    flow, assign, full_variable_count = _indices(supplier_count, transporter_count)

```

```

flow_count = supplier_count * transporter_count
variable_count = full_variable_count if enforce_single_carrier else flow_count
builder = ConstraintBuilder(variable_count)

# 每家供应商一周的预计实际供货不能超过历史估计的常规供货上限。
for i in range(supplier_count):
    builder.add(flow[i], np.ones(transporter_count), -np.inf, capacities[i])
# 每家转运商每周最多承运 6000 立方米原料。
for j in range(transporter_count):
    builder.add(flow[:, j], np.ones(supplier_count), -np.inf, TRANSPORTER_CAPACITY)
if enforce_single_carrier:
    # 问题2把“尽量”落实为可执行硬约束；问题3/4只将其作为披露指标。
    for i in range(supplier_count):
        builder.add(assign[i], np.ones(transporter_count), -np.inf, 1.0)
        for j in range(transporter_count):
            builder.add(
                np.array([flow[i, j], assign[i, j]]),
                np.array([1.0, -capacities[i]]),
                -np.inf,
                0.0,
            )
    lower = np.zeros(variable_count, dtype=float)
    upper = np.concatenate(
        [
            np.repeat(capacities, transporter_count),
            np.ones(flow_count, dtype=float),
        ]
    )
    integrality = np.concatenate(
        [np.zeros(flow_count, dtype=int), np.ones(flow_count, dtype=int)]
    )
else:
    lower = np.zeros(variable_count, dtype=float)
    upper = np.repeat(capacities, transporter_count)
    integrality = np.zeros(variable_count, dtype=int)
product_coefficient = np.array(
    [
        (1.0 - losses[j]) / raw_per_product[i]
        for i in range(supplier_count)
        for j in range(transporter_count)
    ],
    dtype=float,
)
loss_coefficient = np.array(
    [losses[j] for _i in range(supplier_count) for j in range(transporter_count)], dtype=float
)
raw_coefficient = np.ones(supplier_count * transporter_count, dtype=float)
cost_coefficient = np.repeat(unit_cost, transporter_count)
metadata = {
    "flow": flow,
    "assign": assign,
    "capacities": capacities,
    "raw_per_product": raw_per_product,
    "losses": losses,
    "product_coefficient": product_coefficient,
    "loss_coefficient": loss_coefficient,
    "raw_coefficient": raw_coefficient,
    "cost_coefficient": cost_coefficient,
    "single_carrier_hard": np.array([enforce_single_carrier]),
}
return builder, lower, upper, integrality, metadata

def _with_constraint(
    base_builder: ConstraintBuilder,
    indices: np.ndarray,

```



```

        values: np.ndarray,
        lower: float,
        upper: float,
    ) -> ConstraintBuilder:
        """复制当前约束并追加一行，服务词典序目标的后续阶段。"""
        clone = ConstraintBuilder(base_builder.variable_count)
        clone.rows = list(base_builder.rows)
        clone.columns = list(base_builder.columns)
        clone.values = list(base_builder.values)
        clone.lower = list(base_builder.lower)
        clone.upper = list(base_builder.upper)
        clone._row = base_builder._row
        clone.add(indices, values, lower, upper)
        return clone

def _decode_solution(
    data: dict[str, Any],
    candidate_indices: np.ndarray,
    run: MilpRun,
    metadata: dict[str, np.ndarray],
    demand: float,
    part: str,
    selection_method: str,
) -> dict[str, Any]:
    """将单周最优流量扩展为题目要求的24周方案和库存轨迹。"""
    supplier_count = len(candidate_indices)
    transporter_count = len(data["transporter_ids"])
    flow = run.x[metadata["flow"].reshape(-1)].reshape(supplier_count, transporter_count)
    shipment_one_week = np.zeros((len(data["supplier_ids"]), transporter_count), dtype=float)
    shipment_one_week[candidate_indices, :] = flow
    supply_one_week = shipment_one_week.sum(axis=1)
    alpha = data["order_response_ratio"]
    order_one_week = np.divide(
        supply_one_week,
        alpha,
        out=np.zeros_like(supply_one_week),
        where=alpha > TOLERANCE,
    )
    arrival_raw_one_week = (shipment_one_week * (1.0 - metadata["losses"])[None, :]).sum(axis=1)
    arrival_product = float(np.sum(arrival_raw_one_week / data["raw_per_product"]))
    losses_by_transporter = (shipment_one_week * metadata["losses"])[None, :].sum(axis=0)
    active = [
        data["supplier_ids"][index]
        for index in range(len(data["supplier_ids"]))
        if supply_one_week[index] > TOLERANCE
    ]
    inventory = [2.0 * demand]
    for _week in range(HORIZON):
        inventory.append(inventory[-1] + arrival_product - demand)
    return {
        "problem_part": part,
        "demand_product_m3_per_week": demand,
        "selection_method": selection_method,
        "selected_supplier_ids": active,
        "active_supplier_ids": active,
        "candidate_supplier_ids": [data["supplier_ids"][index] for index in candidate_indices],
        "supplier_ids": data["supplier_ids"],
        "material_types": data["material_types"],
        "transporter_ids": data["transporter_ids"],
        "orders_raw_m3": np.repeat(order_one_week[:, None], HORIZON, axis=1).tolist(),
        "expected_supply_raw_m3": np.repeat(supply_one_week[:, None], HORIZON, axis=1).tolist(),
        "shipments_raw_m3": np.repeat(shipment_one_week[:, None, :], HORIZON, axis=1).tolist(),
        "arrivals_raw_m3": np.repeat(arrival_raw_one_week[:, None], HORIZON, axis=1).tolist(),
        "arrivals_product_equivalent_m3": [arrival_product] * HORIZON,
        "losses_by_transporter_raw_m3": np.repeat(losses_by_transporter[None, :], HORIZON,

```

```

axis=0).tolist(),
    "inventory_product_equivalent_m3": inventory,
    "model_metadata": {
        "planned_loss_rate_by_transporter": metadata["losses"].tolist(),
        "regular_order_capacity_raw_m3": data["regular_order_capacity"].tolist(),
        "order_response_ratio": data["order_response_ratio"].tolist(),
        "single_carrier_hard": bool(metadata["single_carrier_hard"][0]),
    },
}

def _objective_values(solution: dict[str, Any], data: dict[str, Any]) -> dict[str, float]:
    supply = np.asarray(solution["expected_supply_raw_m3"], dtype=float)
    shipments = np.asarray(solution["shipments_raw_m3"], dtype=float)
    cost = float(np.sum(supply * data["unit_cost"][:, None]))
    loss = float(np.sum(shipments * data["loss_mean"][None, None, :]))
    raw = float(supply.sum())
    a_count = float(supply[np.array(data["material_types"]) == "A"].sum())
    c_count = float(supply[np.array(data["material_types"]) == "C"].sum())
    return {
        "purchase_cost_relative": cost,
        "transport_loss_raw_m3": loss,
        "total_expected_supply_raw_m3": raw,
        "a_expected_supply_raw_m3": a_count,
        "c_expected_supply_raw_m3": c_count,
    }

def choose_minimum_supplier_set(data: dict[str, Any], demand: float, loss_multiplier: float = 1.0) ->
np.ndarray:
    """按净产品能力从高到低累加，得到满足周需求所需的最少供应商数。

    当每个供应商的选择成本同为1且能力均非负时，该排序累加等价于最小基数覆盖问题的
    精确解；这里使用损耗最小转运商的预测损耗作为筛选阶段的必要条件近似，正式模型再
    用所有转运能力作可行性验证。
    """
    best_loss = float(np.min(np.clip(data["loss_mean"] * loss_multiplier, 0.0, 0.99)))
    ability = data["regular_order_capacity"] * (1.0 - best_loss) / data["raw_per_product"]
    usable = np.flatnonzero(ability > TOLERANCE)
    order = usable[np.argsort(-ability[usable], kind="stable")]
    cumulative = np.cumsum(ability[order])
    position = int(np.searchsorted(cumulative, demand - TOLERANCE))
    if position >= len(order):
        raise RuntimeError("基于常规供货能力无法满足周生产需求")
    return order[: position + 1]

def baseline_problem2(data: dict[str, Any], demand: float = DEMAND_BASE) -> dict[str, Any]:
    """透明基线：按最少能力集合、单位产品采购成本和最低损耗承运商贪心安排。

    它不调用优化器，专门作为正式词典序 MILP 的可解释对照。供应商只分配给一名
    转运商；若其最优承运商剩余能力不足，则该供应商本周不再拆分运输。
    """
    initial_candidates = choose_minimum_supplier_set(data, demand)
    best_loss = float(np.min(data["loss_mean"]))
    ability = data["regular_order_capacity"] * (1.0 - best_loss) / data["raw_per_product"]
    ranked_candidates = np.flatnonzero(ability > TOLERANCE)[np.argsort(-ability[ability > TOLERANCE],
kind="stable")]
    candidates = initial_candidates.copy()
    minimum_count = len(initial_candidates)

    def make_greedy_flow(candidate_array: np.ndarray) -> tuple[np.ndarray, dict[str, np.ndarray],
np.ndarray, float]:
        _builder, local_lower, _upper, _integrality, local_meta = _base_builder(data, candidate_array,
enforce_single_carrier=True)

```

```

        transporter_count = len(data["transporter_ids"])
        local_flow = np.zeros((len(candidate_array), transporter_count), dtype=float)
        remaining_product = demand
        remaining_transport = np.full(transporter_count, TRANSPORTER_CAPACITY, dtype=float)
        effective_cost = data["unit_cost"][candidate_array] * data["raw_per_product"][candidate_array] /
(1.0 - np.min(local_meta["losses"]))
        supplier_order = np.argsort(effective_cost, kind="stable")
        transport_order = np.argsort(local_meta["losses"], kind="stable")
        for local_index in supplier_order:
            if remaining_product <= TOLERANCE:
                break
            supplier_index = candidate_array[local_index]
            for transporter in transport_order:
                if remaining_transport[transporter] <= TOLERANCE:
                    continue
                needed_raw = remaining_product * data["raw_per_product"][supplier_index] / (1.0 -
local_meta["losses"][transporter])
                volume = min(data["regular_order_capacity"][supplier_index], needed_raw,
remaining_transport[transporter])
                if volume <= TOLERANCE:
                    continue
                local_flow[local_index, transporter] = volume
                remaining_transport[transporter] -= volume
                remaining_product -= volume * (1.0 - local_meta["losses"][transporter]) /
data["raw_per_product"][supplier_index]
                break
            return local_flow, local_meta, local_lower, remaining_product

flow, meta, lower, remaining_product = make_greedy_flow(candidates)
for candidate in ranked_candidates:
    if remaining_product <= TOLERANCE:
        break
    if candidate in candidates:
        continue
    candidates = np.append(candidates, candidate)
    flow, meta, lower, remaining_product = make_greedy_flow(candidates)
if remaining_product > TOLERANCE:
    raise RuntimeError(f"透明基线未能满足周需求, 缺口 {remaining_product:.6f}")
vector = np.zeros(len(lower), dtype=float)
vector[meta["flow"].reshape(-1)] = flow.reshape(-1)
pseudo_run = MilpRun(vector, {"solver": "deterministic greedy baseline", "status_text": "feasible"})
solution = _decode_solution(data, candidates, pseudo_run, meta, demand, "2", "最小能力集合上的成本优先
单承运商贪心基线")
solution["selection_minimum_count"] = int(minimum_count)
solution["baseline_selected_count"] = int(len(candidates))
solution["selected_supplier_ids"] = [data["supplier_ids"][index] for index in candidates]
solution["objective"] = _objective_values(solution, data)
solution["baseline_limitations"] = [
    "按供应商顺序贪心, 未在供应商之间全局平衡采购成本与损耗。",
    "使用历史常规供货上限, 不能保证未来每周真实供货。",
]
return solution

def solve_problem2(data: dict[str, Any], demand: float = DEMAND_BASE, loss_multiplier: float = 1.0,
                    cap_multiplier: dict[str, float] | None = None,
                    time_limit_seconds: float = 300.0) -> tuple[dict[str, Any], list[dict[str, Any]]]:
    """问题2: 在全部可用供应商上依次最小化基数、采购成本和运输损耗。"""
    adjusted = dict(data)
    capacities = data["regular_order_capacity"].copy()
    if cap_multiplier:
        for supplier_id, multiplier in cap_multiplier.items():
            try:
                index = data["supplier_ids"].index(supplier_id)
            except ValueError:

```

```

        continue
        capacities[index] *= multiplier
        adjusted["regular_order_capacity"] = capacities
        candidates = np.flatnonzero(adjusted["regular_order_capacity"] > TOLERANCE)
        builder, lower, upper, integrality, meta = _base_builder(adjusted, candidates, loss_multiplier)
        flow_count = len(candidates) * len(data["transporter_ids"])
        flow_indices = np.arange(flow_count)
        assign_indices = meta["assign"].reshape(-1)
        stage0 = _with_constraint(builder, flow_indices, meta["product_coefficient"], demand, np.inf)
        selection_objective = np.concatenate(
            [np.zeros(flow_count), np.ones(len(lower) - flow_count)]
        )
        selection_run = _solve(
            selection_objective,
            stage0,
            lower,
            upper,
            integrality,
            "问题2供应商基数最小化",
            time_limit_seconds,
        )
        minimum_count = int(round(float(selection_run.x[assign_indices].sum())))
        stage1 = _with_constraint(
            stage0,
            assign_indices,
            np.ones(len(assign_indices)),
            minimum_count,
            minimum_count,
        )
        cost_objective = np.concatenate([meta["cost_coefficient"], np.zeros(len(lower) - flow_count)])
        try:
            cost_run = _solve(cost_objective, stage1, lower, upper, integrality, "问题2采购成本最小化",
            time_limit_seconds)
        except OptimizationStatusError as error:
            # 限时中断时保留已证明的基数阶段，避免压力测试丢失可行解的规模口径。
            error.status["locked_supplier_count"] = minimum_count
            error.status["selection_stage"] = selection_run.status
            raise
        cost_limit = float(cost_run.x[:flow_count] @ meta["cost_coefficient"])
        stage2 = _with_constraint(stage1, flow_indices, meta["cost_coefficient"], -np.inf, cost_limit +
        TOLERANCE)
        loss_objective = np.concatenate([meta["loss_coefficient"], np.zeros(len(lower) - flow_count)])
        loss_run = _solve(loss_objective, stage2, lower, upper, integrality, "问题2运输损耗最小化",
        time_limit_seconds)
        solution = _decode_solution(
            adjusted, candidates, loss_run, meta, demand, "2", "全候选空间基数-成本-损耗词典序 MILP"
        )
        selected_local = np.flatnonzero(loss_run.x[meta["assign"]].sum(axis=1) > 0.5)
        selected_indices = candidates[selected_local]
        solution["selection_minimum_count"] = minimum_count
        solution["selected_supplier_ids"] = [data["supplier_ids"][index] for index in selected_indices]
        solution["selection_capacity_product_equivalent_m3"] = float(
            np.sum(adjusted["regular_order_capacity"][selected_indices] * (1.0 - np.min(meta["losses"]))) /
            adjusted["raw_per_product"][selected_indices])
        )
        solution["objective"] = _objective_values(solution, adjusted)
        return solution, [selection_run.status, cost_run.status, loss_run.status]

def solve_problem3(data: dict[str, Any], demand: float = DEMAND_BASE, loss_multiplier: float = 1.0) ->
tuple[dict[str, Any], list[dict[str, Any]]]:
    """问题3: 词典序地少用C、少用原料（自然偏向A）、再少损耗。"""
    candidates = np.flatnonzero(data["regular_order_capacity"] > TOLERANCE)
    builder, lower, upper, integrality, meta = _base_builder(
        data, candidates, loss_multiplier, enforce_single_carrier=False
    )

```

```

)
flow_count = len(candidates) * len(data["transporter_ids"])
flow_indices = np.arange(flow_count)
stage0 = _with_constraint(builder, flow_indices, meta["product_coefficient"], demand, np.inf)
candidate_types = np.array(data["material_types"])[candidates]
c_objective_flow = np.repeat((candidate_types == "C").astype(float), len(data["transporter_ids"]))
c_objective = np.concatenate([c_objective_flow, np.zeros(len(lower) - flow_count)])
c_run = _solve(c_objective, stage0, lower, upper, integrality, "问题3 C类原料最小化")
c_limit = float(c_run.x[:flow_count] @ c_objective_flow)
stage1 = _with_constraint(stage0, flow_indices, c_objective_flow, -np.inf, c_limit + TOLERANCE)
raw_objective = np.concatenate([meta["raw_coefficient"], np.zeros(len(lower) - flow_count)])
raw_run = _solve(raw_objective, stage1, lower, upper, integrality, "问题3原料运输量最小化")
raw_limit = float(raw_run.x[:flow_count] @ meta["raw_coefficient"])
stage2 = _with_constraint(stage1, flow_indices, meta["raw_coefficient"], -np.inf, raw_limit +
TOLERANCE)
loss_objective = np.concatenate([meta["loss_coefficient"], np.zeros(len(lower) - flow_count)])
loss_run = _solve(loss_objective, stage2, lower, upper, integrality, "问题3运输损耗最小化")
loss_run = _decode_solution(data, candidates, loss_run, meta, demand, "3", "C最少-原料量最少-损耗最少的
词典序 LP")
solution["objective"] = _objective_values(solution, data)
statuses = [c_run.status, raw_run.status, loss_run.status]
for status in statuses:
    status["model_type"] = "continuous_transport_relaxation_with_split_diagnostics"
return solution, statuses

def solve_problem4(data: dict[str, Any], loss_multiplier: float = 1.0) -> tuple[dict[str, Any],
list[dict[str, Any]]]:
    """问题4: 在所有可用供应商与转运能力下最大化可持续周产能。"""
    candidates = np.flatnonzero(data["regular_order_capacity"] > TOLERANCE)
    builder, lower, upper, integrality, meta = _base_builder(
        data, candidates, loss_multiplier, enforce_single_carrier=False
    )
    flow_count = len(candidates) * len(data["transporter_ids"])
    flow_indices = np.arange(flow_count)
    # 最大化到货可生产量等价于最小化其相反数; 正系数确保不会无故闲置可用能力。
    objective = np.concatenate([-meta["product_coefficient"], np.zeros(len(lower) - flow_count)])
    run = _solve(objective, builder, lower, upper, integrality, "问题4可持续周产能最大化")
    gross_product = float(run.x[:flow_count] @ meta["product_coefficient"])
    solution = _decode_solution(data, candidates, run, meta, gross_product, "4", "全供应网络产能最大化
LP")
    solution["maximum_weekly_production_m3"] = gross_product
    solution["capacity_increase_m3"] = gross_product - DEMAND_BASE
    solution["capacity_increase_ratio"] = gross_product / DEMAND_BASE - 1.0
    solution["objective"] = _objective_values(solution, data)
    run.status["model_type"] = "continuous_transport_relaxation_with_split_diagnostics"
    return solution, [run.status]

```

18.2.4 B.4 独立复算

文件: code/validator/independent_validator.py; 作用: 不导入求解器地复算目标值、硬约束和 Excel 回填; 输入: 正式决策结果与官方数据; 输出: 约束、目标、模板一致性和故障注入报告。

18.3 附录 C: MATLAB 文件清单

MATLAB 独立复现共 8 个源文件、606 行。所有优化模块直接读取 load_official_data.m 生成的官方数据统计量, 不调用 Python。

表 14 MATLAB 源文件与职责

文件	职责
load_official_data.m	独立读取官方 Excel 与构造参数
evaluate_suppliers.m	四指标评分、排序和权重敏感性
build_flow_model.m	单周线性流模型公共矩阵
solve_problem2.m	intlinprog 最小基数与两阶段 MILP
solve_problem3.m	linprog 三阶段词典序 LP
solve_problem4.m	linprog 最大产能 LP
validate_solution.m	供应、运输、到货、库存约束复核
run_cross_language_validation.m	运行、对照、JSON/CSV 输出

18.4 附录 D: MATLAB 核心源码

以下完整列出三个求解函数和硬约束检查函数；数据加载、矩阵构造与总入口保存在同目录的可复现材料中。

18.4.1 D.1 问题二 intlinprog

文件: solve_problem2.m; 作用: 独立完成最小基数、采购成本和运输损耗三阶段 MILP; 输入: load_official_data.m 产生的官方数据结构; 输出: 供应商选择、运输流、目标值、库存和退出状态。

```
function result = solve_problem2(data)
% SOLVE_PROBLEM2 在全部供应商上独立求解基数、成本和损耗三阶段 MILP。

candidates = find(data.regularCapacity > data.tolerance);
model = build_flow_model(data, candidates, true);
flowIdx = 1:model.flowCount;
assignIdx = model.flowCount + (1:model.flowCount);
A0 = [model.A; sparse(1, flowIdx, -model.productCoefficient, 1, numel(model.lb))];
b0 = [model.b; -data.demand];

% 第一阶段直接在完整运输模型中最小化启用供应商数。
fSelection = zeros(numel(model.lb), 1);
fSelection(assignIdx) = 1;
selectionOptions = optimoptions("intlinprog", "Display", "off", ...
    "RelativeGapTolerance", 0, "AbsoluteGapTolerance", 1e-9, ...
    "ConstraintTolerance", 1e-9, "MaxTime", 300);
tic;
[z, minimumCount, exitflagSelection, outputSelection] = intlinprog( ...
    fSelection, model.intcon, A0, b0, [], [], model.lb, model.ub, selectionOptions);
selectionRuntime = toc;
assert(exitflagSelection > 0, "问题二最小供应商模型未得到最优解");

% 后续阶段只锁定最小基数值, 不固定具体供应商名单。
selectionRow = sparse(1, assignIdx, 1, 1, numel(model.lb));
A1 = [A0; selectionRow; -selectionRow];
b1 = [b0; round(minimumCount); -round(minimumCount)];
costObjective = [model.costCoefficient; zeros(numel(model.lb) - model.flowCount, 1)];
lossObjective = [model.lossCoefficient; zeros(numel(model.lb) - model.flowCount, 1)];
options = optimoptions("intlinprog", "Display", "off", ...
    "RelativeGapTolerance", 0, "AbsoluteGapTolerance", 1e-9, ...
    "ConstraintTolerance", 1e-9, "MaxTime", 300);

tic;
[xCost, costValue, exitflagCost, outputCost] = intlinprog(costObjective, ...
    model.intcon, A1, b1, [], [], model.lb, model.ub, options);
runtimeCost = toc;
assert(exitflagCost > 0, "问题二采购成本阶段未得到最优解");

costRow = sparse(1, flowIdx, model.costCoefficient, 1, numel(model.lb));
A2 = [A1; costRow];
b2 = [b1; costValue + data.tolerance];
tic;
[xLoss, lossValue, exitflagLoss, outputLoss] = intlinprog(lossObjective, ...
```

```

        model.intcon, A2, b2, [], [], model.lb, model.ub, options);
runtimeLoss = toc;
assert(exitflagLoss > 0, "问题二运输损耗阶段未得到最优解");

flow = reshape(xLoss(1:model.flowCount), model.m, model.n)';
result.minimumSupplierCount = round(minimumCount);
assignment = reshape(xLoss(assignIdx), model.m, model.n)';
result.selectionVector = sum(assignment, 2);
result.candidates = candidates;
result.flow = flow;
result.expectedSupply = sum(flow, 2);
result.productArrival = model.productCoefficient' * xLoss(1:model.flowCount);
result.purchaseCost = model.costCoefficient' * xLoss(1:model.flowCount);
result.transportLoss = lossValue;
result.inventory = 2 * data.demand * ones(25, 1);
result.exitflags = [exitflagSelection, exitflagCost, exitflagLoss];
result.runtimes = [selectionRuntime, runtimeCost, runtimeLoss];
result.outputs = {outputSelection, outputCost, outputLoss};
end

```

18.4.2 D.2 问题三 linprog

文件: solve_problem3.m; 作用: 独立完成少 C、少总原料和少损耗三阶段 LP; 输入: 官方数据结构; 输出: 连续运输流、原料结构、库存和退出状态。

```

function result = solve_problem3(data)
%SOLVE_PROBLEM3 用 linprog 独立求解 C类、总原料、损耗三阶段 LP。

candidates = find(data.regularCapacity > data.tolerance);
model = build_flow_model(data, candidates, false);
A0 = [model.A; -model.productCoefficient'];
b0 = [model.b; -data.demand];
options = optimoptions("linprog", "Algorithm", "dual-simplex-highs", ...
    "Display", "off", "ConstraintTolerance", 1e-9, ...
    "OptimalityTolerance", 1e-9, "MaxIterations", 1e6);

candidateTypes = data.materialTypes(candidates);
cObjective = repelem(double(candidateTypes == "C"), model.m);
tic;
[xC, cValue, exitflagC, outputC] = linprog(cObjective, A0, b0, ...
    [], [], model.lb, model.ub, options);
runtimeC = toc;
assert(exitflagC > 0, "问题三 C 类最少阶段未得到最优解");

A1 = [A0; cObjective'];
b1 = [b0; cValue + data.tolerance];
tic;
[xRaw, rawValue, exitflagRaw, outputRaw] = linprog(model.rawCoefficient, ...
    A1, b1, [], [], model.lb, model.ub, options);
runtimeRaw = toc;
assert(exitflagRaw > 0, "问题三总原料最少阶段未得到最优解");

A2 = [A1; model.rawCoefficient'];
b2 = [b1; rawValue + data.tolerance];
tic;
[xLoss, lossValue, exitflagLoss, outputLoss] = linprog(model.lossCoefficient, ...
    A2, b2, [], [], model.lb, model.ub, options);
runtimeLoss = toc;
assert(exitflagLoss > 0, "问题三损耗最少阶段未得到最优解");

flow = reshape(xLoss, model.m, model.n)';
supply = sum(flow, 2);
result.candidates = candidates;
result.flow = flow;
result.expectedSupply = supply;
result.aSupply = sum(supply(candidateTypes == "A"));

```

```

result.bSupply = sum(supply(candidateTypes == "B"));
result.cSupply = sum(supply(candidateTypes == "C"));
result.totalRaw = sum(supply);
result.transportLoss = lossValue;
result.productArrival = model.productCoefficient' * xLoss;
result.exitflags = [exitflagC, exitflagRaw, exitflagLoss];
result.runtimes = [runtimeC, runtimeRaw, runtimeLoss];
result.outputs = {outputC, outputRaw, outputLoss};
end

```

18.4.3 D.3 问题四 linprog

文件: solve_problem4.m; 作用: 独立最大化可持续周产能; 输入: 供应与转运能力、损耗率和原料换算系数; 输出: 最大周产能、运输流、库存和退出状态。

```

function result = solve_problem4(data)
%SOLVE_PROBLEM4 用 linprog 独立最大化损耗后产品等价周到货量。

candidates = find(data.regularCapacity > data.tolerance);
model = build_flow_model(data, candidates, false);
options = optimoptions("linprog", "Algorithm", "dual-simplex-highs", ...
    "Display", "off", "ConstraintTolerance", 1e-9, ...
    "OptimalityTolerance", 1e-9, "MaxIterations", 1e6);
tic;
[x, objective, exitflag, output] = linprog(-model.productCoefficient, ...
    model.A, model.b, [], [], model.lb, model.ub, options);
runtime = toc;
assert(exitflag > 0, "问题四最大产能模型未得到最优解");

flow = reshape(x, model.m, model.n)';
supply = sum(flow, 2);
candidateTypes = data.materialTypes(candidates);
result.candidates = candidates;
result.flow = flow;
result.expectedSupply = supply;
result.aSupply = sum(supply(candidateTypes == "A"));
result.bSupply = sum(supply(candidateTypes == "B"));
result.cSupply = sum(supply(candidateTypes == "C"));
result.totalRaw = sum(supply);
result.transportLoss = model.lossCoefficient' * x;
result.maximumWeeklyCapacity = -objective;
result.productArrival = -objective;
result.exitflag = exitflag;
result.runtime = runtime;
result.output = output;
end

```

18.4.4 D.4 独立硬约束检查

文件: validate_solution.m; 作用: 复核非负性、供应能力、流量平衡、转运能力、到货、生产需求和库存; 输入: MATLAB 独立解与官方数据; 输出: 逐类违反数、最大违反量和业务聚合量。

```

function validation = validate_solution(data, result, problemNumber)
%VALIDATE_SOLUTION 独立复核 MATLAB 决策的供应、运输、损耗和库存约束。

tol = data.tolerance;
candidates = result.candidates(:);
flow = result.flow;
supply = sum(flow, 2);
carrierLoad = sum(flow, 1);
arrivalRaw = sum(flow .* (1 - data.lossMean'), 2);
arrivalProduct = sum(arrivalRaw ./ data.rawPerProduct(candidates));

checks = struct();
checks.nonnegative = sum(flow(:) < -tol);
checks.supplierCapacity = sum(supply > data.regularCapacity(candidates) + tol);
checks.flowSupplyBalance = sum(abs(supply - result.expectedSupply) > tol);

```



```

checks.transporterCapacity = sum(carrierLoad > data.transporterCapacity + tol);
checks.arrivalConsistency = double(abs(arrivalProduct - result.productArrival) > tol);

if problemNumber == 2
    checks.singleCarrier = sum(sum(flow > tol, 2) > 1);
    checks.productionDemand = double(arrivalProduct < data.demand - tol);
    inventory = 2 * data.demand;
    inventoryViolations = 0;
    for t = 1:24
        inventory = inventory + arrivalProduct - data.demand;
        inventoryViolations = inventoryViolations + double(inventory < 2 * data.demand - tol);
    end
    checks.inventory = inventoryViolations;
elseif problemNumber == 3
    checks.singleCarrier = 0;
    checks.productionDemand = double(arrivalProduct < data.demand - tol);
    checks.inventory = 0;
else
    checks.singleCarrier = 0;
    checks.productionDemand = 0;
    checks.inventory = 0;
end

values = struct2array(checks);
validation.checks = checks;
validation.totalHardViolations = sum(values);
validation.arrivalProduct = arrivalProduct;
validation.carrierLoad = carrierLoad;
validation.maxViolationMagnitude = max([0; ...
    -min(flow(:)); ...
    max(supply - data.regularCapacity(candidates)); ...
    max(carrierLoad' - data.transporterCapacity)]);
end

```

18.5 附录 E: Python–MATLAB 结果交叉验证

表 15 双语言评价与问题二对照

比较项	Python	MATLAB	绝对误差
前 10 排名	完全一致	完全一致	0
前 50 排名	完全一致	完全一致	0
前 50 得分最大误差	-	-	1.11×10^{-16}
权重敏感性最大误差	-	-	4.44×10^{-16}
最少供应商数	26	26	0
24 周采购成本	491125.620530536	491125.620530303	2.32×10^{-7}
24 周运输损耗/m ³	2470.51643049995	2470.51643050434	4.39×10^{-9}
期末库存/m ³	56400.0000003206	56400	3.21×10^{-7}

表 16 双语言问题三、四与约束对照

比较项	Python	MATLAB	绝对误差
问题三 A 类/24 周	207057.104839661	207057.104839661	0
问题三 B 类/24 周	157616.966600894	157616.966600894	0
问题三 C 类/24 周	69507.8406208139	69507.8406208138	1.60×10^{-10}
问题三总原料/24 周	434181.912061369	434181.912061369	1.16×10^{-10}
问题三损耗/24 周	2399.75288136428	2399.75288136428	9.09×10^{-13}
问题四周产能	33335.925461873	33335.925461873	2.18×10^{-11}
问题二至四硬约束	0	0	0

评分容差预设设为 10^{-9} ，连续量绝对误差容差预设设为 10^{-6} 。表 15、16 所有项目均通过，最大跨语言误差为 $3.206077796311 \times 10^{-7}$ 。离散模型没有用逐单元决策相等作为通过条件，而是要求供应商数量、目标值、最优性状态和硬约束一致。

18.6 附录 F：复现环境与运行命令

表 17 双语言复现环境

项目	记录
Python	SciPy 1.17.0; HiGHS; 正式入口 <code>python code/run_training.py</code>
Python 源码	14 个文件, 4,151 行
MATLAB	24.1.0.2537033 (R2024a), PCWIN64
Optimization Toolbox	24.1; <code>intlinprog</code> 与 <code>linprog</code> 可用
MATLAB 源码	8 个文件, 606 行
MATLAB 命令	<code>matlab -batch "addpath('code/matlab'); run_cross_language_validation(pwd)"</code>
MATLAB 退出状态	0
MATLAB 总运行时间	27.2286188 s
问题二阶段时间	0.6372922 / 4.8924499 / 16.6143945 s
问题三阶段时间	0.0991937 / 0.0762684 / 0.0205662 s
问题四时间	0.0216611 s

运行记录、逐项对照表和完整源文件分别保存在机器可读 JSON、CSV 与代码目录中。论文附录保留核心源码, 不包含临时调试代码或本地绝对路径。